

ENGINEERING · LARGE LANGUAGE MODELS

The Economics of Inference

Why most production LLM stacks are paying two to ten times what they need to — and the technical levers that close the gap.

An advanced engineering article on fine-tuning, serving, and architectural cost reduction.

BY

Chandra Teja Kokkonda · Pavan Sai Alapati · Pavan Raj Kotagiri · Jagruth Reddy Palle

I. The Bill Nobody Reads

Somewhere in a finance dashboard at most companies running large language models in production, there is a line item that has quietly become one of the largest variable costs in the business. It does not look like compute. It does not look like SaaS. It looks like a vendor bill that grows linearly with traffic, and unlike storage or bandwidth, it does not get cheaper with scale unless someone deliberately makes it cheaper. The default trajectory of an LLM-powered product is for its cost curve to mirror its adoption curve almost exactly, multiplied by whatever model the team picked when the prototype shipped.

This is, in a sense, a remarkable failure of engineering discipline. The same teams who would never deploy an unbounded SQL query into production, or run an $O(n^2)$ loop without profiling, routinely ship LLM features whose unit cost is two to ten times what it needs to be, simply because the prompt was written for the largest available model on the first attempt and the question of right-sizing was never revisited. The good news is that the cost surface of an LLM stack is unusually rich. There are at least a dozen orthogonal levers, almost all of which are cheap to pull, several of which are essentially free, and a handful of which compound multiplicatively. A serious cost program does not require sacrificing quality; in most cases the optimized stack is strictly faster, cheaper, and better than the unoptimized one, because the discipline of compression forces a clarity about the task that the original system never had.

The rest of this article is an attempt to give that program technical depth. It assumes the reader is comfortable with transformer internals, basic numerical-precision tradeoffs, and the working vocabulary of fine-tuning and inference. It is not a survey; it is a working engineer's view of which levers matter, why they matter, and in what order to pull them.



II. What You Are Actually Paying For

Begin with the equation. Per-request cost on a modern serving stack decomposes, to first order, as:

$$C = T_{in} \cdot p_{in} + T_{out} \cdot p_{out} + \alpha \cdot T_{kv} + \beta \cdot idle$$

Where T_{in} and T_{out} are input and output tokens, p_{in} and p_{out} are their respective prices, T_{kv} is the cached prefix length consumed at a discounted

rate, and $\beta \cdot \text{idle}$ captures the amortized cost of underutilized accelerators. The first observation worth internalizing is that output tokens are typically priced at three to five times the input rate, because each output token requires a full forward pass through the model in serial, while input tokens are processed in parallel during the prefill phase. The second is that idle GPU time is not free; it is in fact the most expensive form of compute a team can buy, because it is paid for at on-demand rates and produces nothing.

Below this surface arithmetic sits a more fundamental constraint: LLM inference at the small batch sizes typical of interactive workloads is **memory-bandwidth-bound**, not compute-bound. A 70B-parameter model in bfloat16 is roughly 140 GB of weights. Generating a single token requires reading those weights from HBM through the streaming multiprocessors. On an H100 with around 3 TB/s of memory bandwidth, that read alone takes about 47 milliseconds, which sets a hard ceiling of roughly 21 tokens per second on single-stream decoding — regardless of how many TFLOPS the GPU technically possesses. This is the **roofline** of the system, and almost every inference-side optimization in this article is, viewed at the right altitude, an attempt to push the workload toward the compute-bound regime where the silicon is actually utilized.

That reframing matters because it changes which optimizations are mechanically meaningful. Quantization is not interesting because it reduces FLOPs — it does not, at runtime — but because it reduces the bytes that must traverse the memory hierarchy. Batching is not interesting because it parallelizes computation in some abstract sense, but because it amortizes a single weight-fetch across many concurrent decoding streams, dragging the workload up the roofline. Caching is not interesting because it avoids repeated arithmetic, but because it avoids repeated bandwidth consumption. Once the bandwidth lens is in place, the rest of the optimization landscape becomes legible.



III. Fine-Tuning Without the Capital Expenditure

Full fine-tuning of a modern open-weight model — updating every parameter at full precision with a state-of-the-art optimizer — is, for most enterprise tasks, a category error. It is the technique a researcher reaches for when the question is

whether a behavior is achievable at all; it is rarely the technique an engineer should reach for when the question is whether a behavior can be deployed economically. The arithmetic is straightforward. A 70B model in bfloat16 requires roughly 140 GB for weights, another 140 GB for gradients, and — with the Adam family of optimizers — an additional 280 GB for the two moment estimates in float32. That is well over half a terabyte of activated memory before a single activation tensor is allocated, which forces multi-node tensor-parallel training and the corresponding interconnect costs.

Parameter-efficient fine-tuning collapses this footprint. The central observation, due to Hu et al. in the original LoRA paper, is that the update matrix ΔW learned during adaptation is empirically low-rank: it can be approximated by the product of two thin matrices $B \in \mathbb{R}^{(d \times r)}$ and $A \in \mathbb{R}^{(r \times k)}$ with $r \ll \min(d, k)$, typically $r \in \{4, 8, 16, 32\}$. The forward pass becomes:

$$h = W_\theta x + (B A) x = W_\theta x + \Delta W x$$

W_θ is frozen; only A and B are trained. For a typical 7B model with $r = 8$ applied to the query and value projections, the trainable parameter count drops from billions to a few million — three to four orders of magnitude — while downstream task quality on supervised fine-tuning typically lands within a percentage point of full fine-tuning. The optimizer state shrinks proportionally, since only the LoRA matrices need their moments tracked. The result is that a fine-tune which previously required an eight-GPU node fits, end-to-end, on a single accelerator.

QLoRA and the path to a single 24 GB card

QLoRA, introduced by Dettmers et al. in 2023, takes LoRA further by quantizing the frozen base model to a 4-bit data type called **NF4** (normal float 4), which is information-theoretically near-optimal for normally distributed weights. Crucially, the LoRA adapters themselves remain in bfloat16, so the trainable path retains full numerical fidelity. Two additional tricks — **double quantization**, which compresses the quantization constants themselves, and **paged optimizers**, which use unified memory to spill optimizer state to host RAM when GPU memory pressure spikes — make it possible to fine-tune a 65B-parameter model on a single 48 GB GPU, and a 7B model on a 16 GB consumer card. In published benchmarks the quality gap to a full-precision LoRA run is small and often inside the noise of the eval itself.

The practical upshot is that QLoRA should be the default fine-tuning recipe for almost all enterprise adaptation work in 2026. Full fine-tuning should be

reserved for one of three cases: a research context where reproducing a result requires it, a deployment context where the adapter merging path is unavailable (rare with modern tooling), or a quality regression on a real evaluation set that can be reproduced and traced to the rank or precision of the adaptation. The third case is unusual but real — heavily reasoning-oriented continued pre-training, for instance, sometimes benefits from higher rank or 16-bit adapters.

Beyond LoRA: DoRA, IA3, and the rank-zero regime

Several refinements of LoRA have appeared and are worth knowing. **DoRA** (Weight-Decomposed Low-Rank Adaptation) decomposes each weight matrix into a magnitude vector and a directional component, applying LoRA only to the direction. It closes most of the remaining gap to full fine-tuning, particularly at low ranks, at the cost of a small increase in trainable parameters and training-time overhead. **IA3** scales activations rather than learning additive updates, producing the smallest footprint of any popular PEFT method — typically a few thousand parameters per layer — and is useful for highly constrained deployments or for fine-tuning many specialized variants on top of a single base. **Prefix tuning** and **prompt tuning** train a small set of continuous "soft prompt" embeddings; they are extremely cheap but generally underperform LoRA on complex tasks and are now mostly of historical interest.

The data axis is at least as important as the algorithm

A persistent finding across both academic and industrial fine-tuning is that data curation dominates technique. The LIMA paper demonstrated that a 65B model fine-tuned on 1,000 carefully curated examples could match the apparent quality of one trained on millions of crowd-sourced examples, at less than a tenth of a percent of the cost. The mechanism is not surprising in retrospect: large pre-trained models already contain the capability; fine-tuning is largely a question of *eliciting* it along the right axis, and most large datasets contain redundancy, near-duplicates, and label noise that actively interferes with that elicitation.

Several practices follow. Deduplicate aggressively at the document and near-document level, using MinHash-LSH or embedding-based clustering. Decontaminate against the evaluation set, including paraphrased forms. Apply difficulty-aware sampling: discard examples the base model already handles correctly under simple prompting, and oversample the failure modes you actually need to repair. Use a strong teacher model to generate synthetic data

for narrow tasks, then filter the synthetic output through a quality classifier — this is essentially data distillation, and is now the dominant paradigm for producing small specialized models. Finally, train for one to three epochs and stop. Large models memorize within a single pass on small datasets; further epochs almost always damage generalization.



IV. Serving: Where the Real Money Goes

If fine-tuning is a capital expenditure, inference is the operating one — and at any nontrivial usage level, operating cost dwarfs capital. The optimization opportunities here are deeper, more numerous, and more compoundable than on the training side. They divide cleanly into four families: precision optimizations, which reduce memory bandwidth per token; attention optimizations, which reduce the cost of the KV cache; scheduling optimizations, which raise effective batch size and therefore hardware utilization; and decoding optimizations, which reduce the number of forward passes per generated token.

Precision: weight-only quantization is essentially free quality

Post-training weight quantization compresses model weights to a lower-precision data type — typically INT8, INT4, or FP8 on supported hardware — after the model is already trained. Unlike training quantization, this is a one-shot transformation with no recurring cost, and because inference is memory-bandwidth-bound at small batch sizes, the speedup is roughly proportional to the compression ratio. The dominant methods in production use today are **GPTQ**, which performs layer-wise reconstruction with second-order information; **AWQ** (Activation-aware Weight Quantization), which protects the most salient weight channels based on observed activation magnitudes; and **SmoothQuant**, which redistributes the quantization difficulty between weights and activations to enable full INT8 end-to-end.

Empirically, INT8 weight-only quantization is essentially lossless on most decoder-only language models, with perplexity changes typically below 0.5%. INT4 introduces a small but measurable degradation — often 1–3% on reasoning

benchmarks and negligible on classification — and is the right default for high-throughput production serving where the cost reduction is roughly fourfold. Below 4 bits the picture becomes more delicate: 2- and 3-bit techniques (SpQR, QuIP#, OmniQuant) work, but the quality cliff depends heavily on the specific model and the specific evaluation, and the engineering cost of validating them is rarely worth the marginal saving over INT4.

A subtlety often missed: weight-only quantization helps the bandwidth-bound regime but does *not* help the compute-bound regime that emerges at large batch sizes. If a serving deployment is already saturating tensor cores with concurrent requests, switching from bfloat16 to INT4 may yield disappointingly little speedup because the bottleneck has moved. **Weight-and-activation quantization** (W8A8, FP8) is the right move in that regime, and is now well supported on Hopper-class accelerators.

Format	Memory vs FP16	Speedup	Quality impact
INT8 weight-only	0.50×	1.5–2.0×	Negligible (<0.5% perplexity)
INT4 (AWQ/GPTQ)	0.25×	2.5–4.0×	Small (1–3% on reasoning)
FP8 (W8A8 on Hopper)	0.50×	1.5–2.0×	Near-lossless including at large batch
2–3 bit (experimental)	<0.20×	Hardware-limited	Model- and task-dependent

Attention: paying for context twice is optional

In a vanilla transformer, each token attends to every previous token, and the keys and values for those previous tokens must be cached in GPU memory across the entire generation. The KV cache grows as $O(L \cdot n_{\text{layers}} \cdot n_{\text{heads}} \cdot d_{\text{head}})$ per request and rapidly becomes the dominant memory consumer for long-context workloads — at 32K context on a 70B model, the KV cache can exceed the model weights themselves. Three lines of work have made this manageable.

First, architectural changes to the model itself. **Multi-Query Attention** (MQA) and the more practical **Grouped-Query Attention** (GQA) share keys and values across groups of query heads, reducing the KV cache by factors of four to

eight with minimal quality impact. Almost every modern open-weight model — Llama-class, Qwen-class, Mistral-class — now ships with GQA by default. For self-hosted serving this is essentially free.

Second, kernel-level optimizations. **FlashAttention** and its successors (FlashAttention-2, FlashAttention-3) reorder the attention computation to avoid materializing the full attention matrix in HBM, instead streaming tiles through SRAM. The arithmetic is identical to standard attention; the memory access pattern is dramatically better. The result is 2–4× speedup on the attention computation itself, particularly at long context lengths, and is now table stakes in any serious serving stack.

Third, memory management at the system level. **PagedAttention**, introduced with vLLM, treats the KV cache like virtual memory: it allocates the cache in fixed-size blocks (analogous to OS pages), tracks them with a block table, and shares blocks across requests with identical prefixes via copy-on-write. The practical effect is twofold: memory fragmentation drops from typical 60–80% utilization to over 95%, which means a serving deployment can hold more concurrent requests in flight; and shared system prompts become free, because the blocks containing them are physically deduplicated. For any deployment with a long, stable system prompt — which describes most enterprise LLM applications — this is the single highest-leverage serving optimization available today.

Scheduling: continuous batching as the load-bearing primitive

Naïve serving processes one request at a time, sequentially. Static batching groups requests into fixed-size windows, which raises throughput but punishes latency: every request waits for the slowest member of its batch to finish. **Continuous batching** — also called in-flight batching or iteration-level scheduling — operates at the granularity of single decoding steps. At each step, completed sequences are evicted from the batch and waiting requests are inserted in their place. The forward pass always runs at maximum effective batch size, while short requests escape as soon as they finish.

Empirically, continuous batching delivers 5–20× higher throughput than naïve serving at comparable latencies, and is the single largest reason that modern self-hosted stacks (vLLM, TensorRT-LLM, SGLang, TGI) can approach the unit economics of hosted APIs. If you are running an inference deployment without

continuous batching in 2026, you are not optimizing — you are subsidizing your hardware vendor.

Decoding: getting more tokens per forward pass

The final family of inference optimizations attacks the autoregressive bottleneck itself. **Speculative decoding** uses a small, fast "draft" model to propose k future tokens in a single forward pass; the large "target" model then verifies these tokens in parallel by running a single forward pass over the proposed sequence and comparing logits. Accepted tokens are committed; the first rejected token rolls back the suffix. Because the target model's output distribution is preserved exactly, this is a pure throughput win when the draft model is well-aligned with the target — typical acceptance rates of 60–80% yield $1.5\text{--}3\times$ end-to-end speedup with no quality change.

Newer variants extend the idea. **Medusa** trains additional decoding heads on the target model itself, eliminating the draft model entirely. **EAGLE** and **EAGLE-2** improve draft quality by drafting in feature space rather than token space and verifying along a tree of candidate continuations. **Lookahead decoding** generates parallel disjoint n -grams that are later refined into accepted sequences. The choice between these is a matter of integration cost and the specific traffic pattern; the meta-point is that single-stream decoding throughput, which was once treated as a hard limit, is now an engineering choice.



V. Systemic Leverage: Routing, Caching, and Distillation

The optimizations above operate within a single model. The next family operates across models and across time, and tends to deliver the largest order-of-magnitude wins because it changes which forward passes happen at all.

Routing: most traffic does not deserve your best model

In essentially every production LLM workload that has been carefully measured, the empirical difficulty distribution of requests is heavy-tailed: 80–95% of traffic is handled comfortably by a model an order of magnitude smaller than the one originally deployed, and only a long tail genuinely benefits from the

largest available model. A router is the component that exploits this distribution. The simplest viable router is a small classifier — a fine-tuned DeBERTa-class model or even a logistic regression over prompt features — that predicts which tier of model will succeed on a given input and dispatches accordingly. More sophisticated systems use **cascading**: try the small model first, then escalate to the larger one if a verifier flags low confidence or the output fails a structured-validation check.

Routing is unusually high-leverage because it compounds with every other inference optimization. A $5\times$ reduction from quantization applied to the 90% of traffic that already goes to a smaller, cheaper, quantized model multiplies through to roughly $25\times$ versus the original baseline. The implementation cost is modest — a serviceable router can be built in a week — and the ongoing maintenance is a single classifier retrained quarterly. The hardest part is cultural: persuading product teams to accept that the model they originally specified is not the model their users actually need.

Caching, in three layers

Caching is the rare optimization that delivers free latency and free cost simultaneously. Three layers are worth implementing in sequence.

The **exact-match cache** hashes the prompt and serving parameters and returns the prior response if the hash hits. Trivial to implement, surprisingly effective: on classification, moderation, templated generation, and tool-augmented workflows, hit rates of 10–40% are common. The implementation is a Redis or RocksDB instance keyed on a stable hash of (prompt, model, temperature, system_prompt). The only operational concern is invalidation: if a system prompt changes, the cache must be partitioned or purged.

The **semantic cache** embeds the prompt and looks up nearest neighbors in a vector index; a match above a calibrated similarity threshold returns the cached response. This works well on conversational and FAQ-style traffic where users ask the same thing in different words, but requires careful threshold tuning to avoid serving subtly wrong answers from near-but-not-equivalent queries. The right discipline is to validate the threshold against a held-out eval that includes adversarial near-misses, and to log every cache hit for periodic audit.

The **KV-cache layer**, increasingly exposed by hosted providers as "prompt caching" and by self-hosted stacks via PagedAttention, deduplicates the prefill cost of shared prefixes. Restructuring a prompt so that the static portion —

instructions, schemas, retrieved documents — sits at the front rather than interleaved with user-specific content can convert what would have been a recurring high-cost prefill into an essentially free hit. For applications with system prompts longer than a few thousand tokens, this single change typically reduces total input-token cost by 60–90%.

Distillation: paying once to never pay again

Distillation is the practice of training a small student model to imitate a larger teacher. In the LLM era, this most often takes the form of using a frontier model to generate high-quality outputs on a target task distribution, then supervised fine-tuning a smaller open-weight model on those outputs. The teacher's inference cost is paid once during the data generation phase; the student's inference cost is paid forever, but at a dramatically lower rate.

For narrow, well-defined tasks — structured extraction, in-domain classification, format conversion, routing, summarization within a stable domain — a well-distilled student that is 10–50× smaller than its teacher typically retains 90–99% of the teacher's task-specific quality. The unit-cost reduction is often more than an order of magnitude. The technique generalizes poorly to broad, open-ended capabilities; do not distill general reasoning, but do distill the narrow workflows that account for the majority of production volume.

Two refinements matter. **On-policy distillation** — generating training data from the student's own outputs, scored or corrected by the teacher — closes the exposure bias gap that pure offline distillation can introduce. **White-box distillation**, when teacher and student come from the same family, can match logits directly rather than only sampled outputs, extracting more signal per teacher token; this is what produces the strongest small models in modern open-weight releases.



VI. The Quiet Case for Buying the Server

Every optimization in the preceding sections assumes hardware exists to be optimized on. The question of who owns that hardware is often skipped over, but the arithmetic deserves a closer look. A high-end accelerator rented on demand from a hyperscaler typically costs several dollars per GPU-hour. The same

accelerator, purchased outright and amortized over three years of useful life, costs roughly a dollar per GPU-hour in hardware alone; add electricity, cooling, network, and a fraction of a sysadmin's time and the all-in figure is still well under two. The cloud premium for a steady-state workload — before any reserved-instance discounts are negotiated — typically lands between 2× and 5×, and at peak on-demand pricing can exceed 10×.

That gap exists because the cloud's business model is precisely to absorb the burst: to keep capacity available at a moment's notice and let it sit idle most of the time, with the customer paying for the privilege. For workloads that are themselves bursty — occasional fine-tuning runs, unpredictable evaluation harnesses, batch jobs that run twice a quarter — that pricing is rational and worth paying. For workloads that are not bursty — production inference running around the clock at high utilization, an internal copilot serving thousands of employees, a sustained continued-pre-training schedule — paying cloud rates is paying for flexibility that is not being used.

The objection that on-premise hardware is "operationally complex" deserves more skepticism than it usually receives. A single rack with one or two GPU servers, sitting in a properly cooled utility closet at the office or in a small colocation cage, running a mature serving stack like vLLM or TensorRT-LLM behind a reverse proxy, is not a research project. It is a piece of infrastructure that one competent engineer can operate, that does not require dedicated data-center expertise, and that pays for itself within months at any non-trivial sustained load. The genuine constraints — strict data-residency rules, true variable load, the need for the very latest hardware that has not yet hit the secondary market — are real but narrower than the default discourse suggests.

The pragmatic pattern that has emerged among teams running serious LLM workloads is a hybrid one: own the steady-state, rent the burst. Serve the predictable production traffic on owned hardware; spill into the cloud for occasional spikes, evaluation runs, and access to hardware tiers that have not yet been bought. Use hosted APIs for the long tail of low-volume use cases and for the closed frontier models that are not available any other way. Treat the cloud GPU bill as a variable line item that should track usage spikes, not a fixed line item that grows linearly with the team. The teams that get this right tend to spend something like a third of what their peers do for the same actual computational throughput.



VII. The Cheapest Forward Pass Is the One You Skip

Every technique above operates within a frame that takes the basic system architecture as given. The largest cost reductions, however, often come from changing that frame. Two architectural pivots recur often enough to deserve explicit treatment.

Retrieval instead of memorization

A common antipattern is to fine-tune a model on a corpus of organizational knowledge — product documentation, policy documents, codebases — with the goal of making the model "know" that material. This is almost always the wrong move on cost grounds and frequently the wrong move on quality grounds as well. The fine-tune is expensive to produce, expensive to update (a single new document means another fine-tune), opaque to audit, and prone to confidently hallucinating because the model cannot distinguish memorized fact from memorized noise.

Retrieval-augmented generation inverts the architecture: keep the model small and general; keep the knowledge in an external index; fetch the relevant snippets at query time and condition generation on them. The knowledge layer becomes a vector database or hybrid index, updated incrementally as documents change. The model layer becomes a stable, cheap-to-serve component that does not need to be retrained when the underlying facts change. The cost profile shifts from periodic large fine-tuning runs plus high-token-count prompts to small per-query retrieval costs plus modest grounded prompts — typically a net reduction of an order of magnitude, with the additional benefit that the system can cite its sources and refuse confidently when retrieval returns nothing.

Tools, agents, and the discipline of decomposition

For tasks involving arithmetic, structured lookups, code execution, or interaction with external systems, function calling and tool use almost always dominate pure LLM approaches on cost. A small model with reliable tool use can outperform a frontier model attempting to perform the same operations in its head, at a fraction of the per-request cost, with the additional property that the tool calls produce auditable, deterministic intermediate state. Agentic systems — chains of model calls coordinated around a task — make the cost-quality

tradeoff explicit at each step. A well-designed agent uses the cheapest viable model at each node, escalates only when local confidence is low, and caches intermediate results aggressively. The discipline of decomposition is, in cost terms, the discipline of making sure no token is spent on a step that did not need a token.



VIII. Practice

None of the techniques in this article are individually exotic. Every one is implemented in mature open-source tooling or exposed by major hosted providers. The competitive advantage, as is usually the case in production engineering, lies in their integration and their measurement. A team that ships an instrumented router, prompt caching, continuous batching, INT4 weight quantization on its self-hosted models, and QLoRA as its default fine-tuning recipe — within a quarter, with no exotic infrastructure — will typically find that its LLM bill drops by 70–90% against the unoptimized baseline, with quality, by every honest measurement, holding flat or improving.

The remaining cost should be the cost that reflects the value the system actually creates for its users. Anything more is rent paid to inefficient defaults. The argument of this article is not that LLMs are expensive. It is that the levers are there. Most production stacks have simply not pulled them.

The optimized stack is not the one that runs the smallest model. It is the one where every token is accounted for, and every forward pass is the one that needed to happen.

Appendix. A Cascade Router You Can Steal

Section V argued that routing easy traffic to a cheaper model is the single highest-leverage cost reduction available on the serving side. We did not, at the time, say how to do it. The basic idea is trivial — call the cheap model first, escalate to the expensive one only when needed. The difficulty hides in a question almost every team answers by guessing: *when, exactly, should the router escalate?*

The honest answer is that the escalation threshold is a calibration target, not a hyperparameter to be intuited. For each candidate threshold, run a held-out evaluation set through the router, record the cost and the quality, and pick the cheapest threshold that still meets the quality bar. The loop fits on one screen, and is the thing almost everyone skips:

```
def calibrate(router, eval_set, target_quality=0.95):
    """Sweep thresholds; return the cheapest one that hits the quality bar."""
    best_t, best_cost = None, float("inf")
    for t in [0.50, 0.60, 0.70, 0.75, 0.80, 0.85, 0.90, 0.95]:
        router.threshold = t
        results = [router.route(c.prompt) for c in eval_set]
        quality = sum(c.check(r.text) for c, r in zip(eval_set, results)) /
len(eval_set)
        cost = sum(r.cost_usd for r in results)
        if quality >= target_quality and cost < best_cost:
            best_t, best_cost = t, cost
    return best_t # pin this in production
```

Run once against a representative eval set of fifty to two hundred cases. Pin the resulting threshold in production. Re-run the calibration whenever the traffic distribution shifts materially — quarterly is usually enough.

The full implementation accompanying this article — `cascade_router.py` — is roughly two hundred lines of standard-library Python with no proprietary dependencies. It includes the router class, the calibration routine above, and ready-made adapters for the major hosted APIs. The default confidence signal is the geometric mean of token-level log-probabilities over the first eight decoded tokens, which most major providers expose; for providers that do not, the router accepts a pluggable confidence function, with self-consistency as the obvious

fallback. The file is released into the public domain — copy it, modify it, ship it. No attribution required.

A note on what this is and is not. It is the highest-leverage half-day of engineering work most teams will do this quarter. It is not a substitute for the rest of the optimizations in this article — quantization, prompt caching, continuous batching, and parameter-efficient fine-tuning compose with the cascade rather than competing with it. The point is that the boundary between research code and production code, for a technique this widely cited and this rarely implemented well, is the small piece of code above. Most of the published literature on cascading and model routing assumes this calibration step exists; almost none of the production deployments have it.

References

Key papers behind the techniques discussed above, grouped by the section in which each first appears. arXiv identifiers are given where applicable; most of these papers also have peer-reviewed versions at NeurIPS, ICML, ICLR, or OSDI/SOSP.

PARAMETER-EFFICIENT FINE-TUNING

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS 2023. arXiv:2305.14314.
- Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. (2024). *DoRA: Weight-Decomposed Low-Rank Adaptation*. ICML 2024. arXiv:2402.09353.
- Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., & Raffel, C. (2022). *Few-Shot Parameter-Efficient Fine-Tuning is Better and Cheaper than In-Context Learning* (IA3). arXiv:2205.05638.
- Li, X. L., & Liang, P. (2021). *Prefix-Tuning: Optimizing Continuous Prompts for Generation*. ACL 2021. arXiv:2101.00190.
- Lester, B., Al-Rfou, R., & Constant, N. (2021). *The Power of Scale for Parameter-Efficient Prompt Tuning*. EMNLP 2021. arXiv:2104.08691.
- Zhou, C., Liu, P., Xu, P., Iyer, S., Sun, J., Mao, Y., Ma, X., Efrat, A., Yu, P., Yu, L., et al. (2023). *LIMA: Less Is More for Alignment*. NeurIPS 2023. arXiv:2305.11206.

QUANTIZATION

- Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. arXiv:2210.17323.
- Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., Gan, C., & Han, S. (2023). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration*. arXiv:2306.00978.
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2022). *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models*. arXiv:2211.10438.

Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoefler, T., & Alistarh, D. (2023). *SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression*. arXiv:2306.03078.

ATTENTION AND MEMORY

Shazeer, N. (2019). *Fast Transformer Decoding: One Write-Head is All You Need* (Multi-Query Attention). arXiv:1911.02150.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*. arXiv:2305.13245.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Ré, C. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022. arXiv:2205.14135.

Dao, T. (2023). *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. arXiv:2307.08691.

Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., & Dao, T. (2024). *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision*. NeurIPS 2024. arXiv:2407.08608.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention* (vLLM). SOSP 2023. arXiv:2309.06180.

DECODING AND SCHEDULING

Leviathan, Y., Kalman, M., & Matias, Y. (2023). *Fast Inference from Transformers via Speculative Decoding*. ICML 2023. arXiv:2211.17192.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., & Jumper, J. (2023). *Accelerating Large Language Model Decoding with Speculative Sampling*. arXiv:2302.01318.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., & Dao, T. (2024). *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. arXiv:2401.10774.

Li, Y., Wei, F., Zhang, C., & Zhang, H. (2024). *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*. ICML 2024. arXiv:2401.15077.

Fu, Y., Bailis, P., Stoica, I., & Zhang, H. (2024). *Break the Sequential Dependency of LLM Inference Using Lookahead Decoding*. arXiv:2402.02057.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., & Chun, B.-G. (2022). *Orca: A Distributed Serving System for Transformer-Based Generative Models* (continuous batching). OSDI 2022.

MULTI-TENANT SERVING

Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., Gonzalez, J. E., & Stoica, I. (2023). *S-LoRA: Serving Thousands of Concurrent LoRA Adapters*. arXiv:2311.03285.